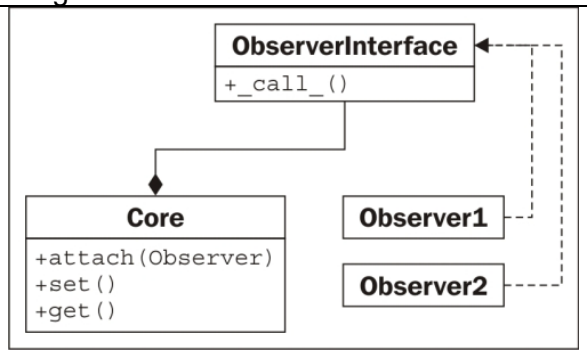
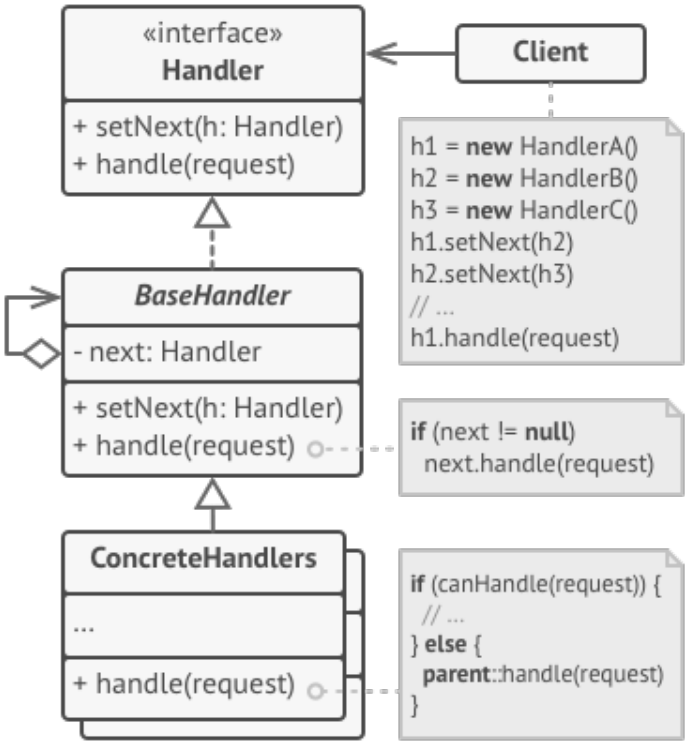
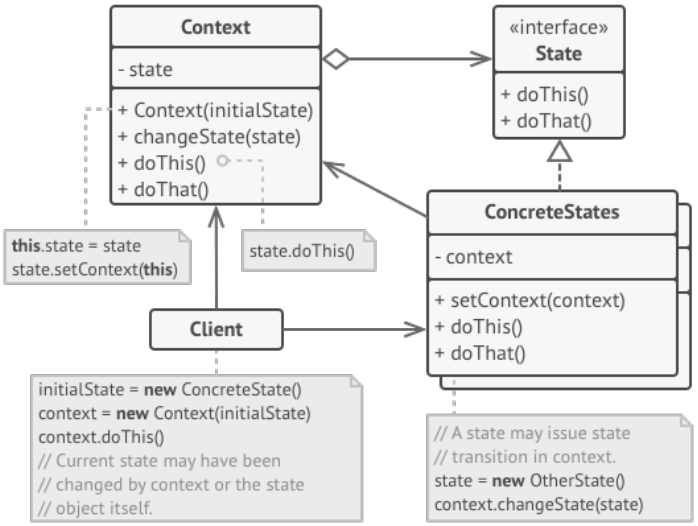
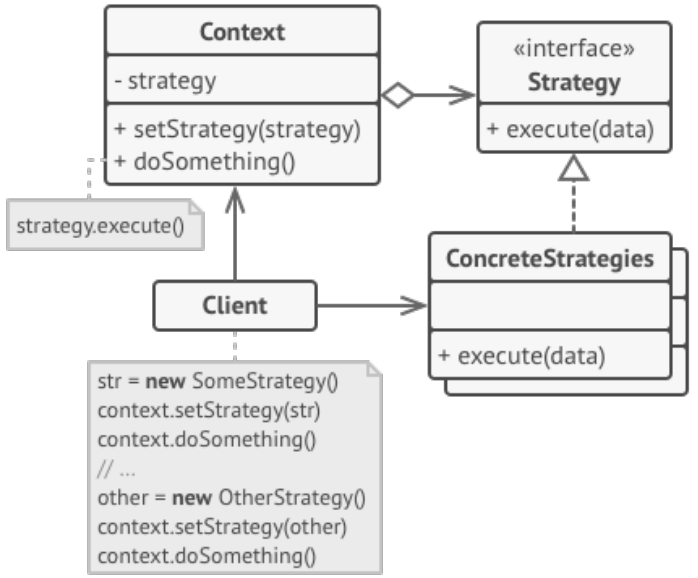
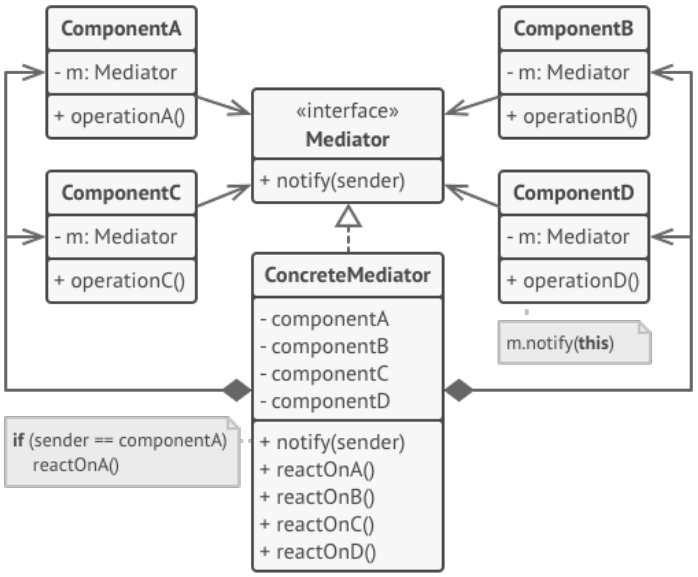
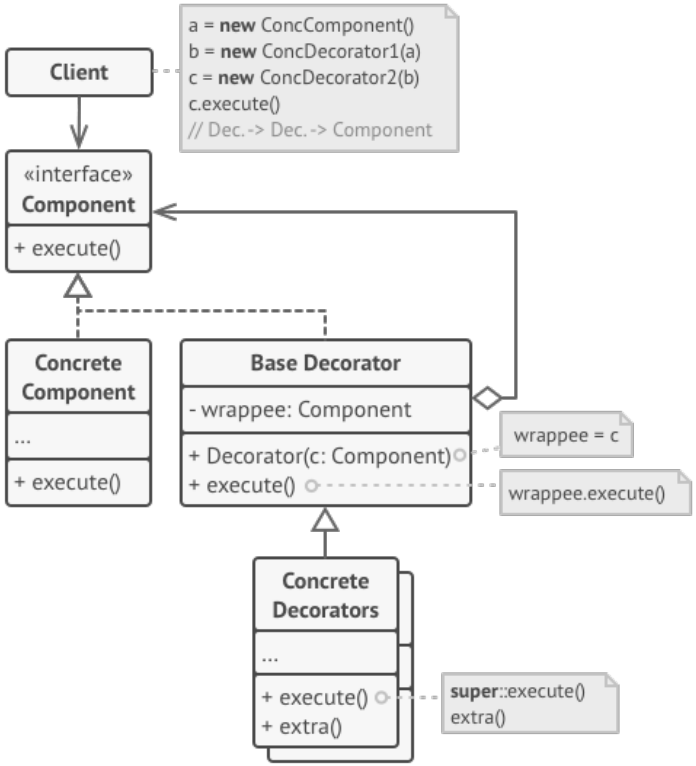
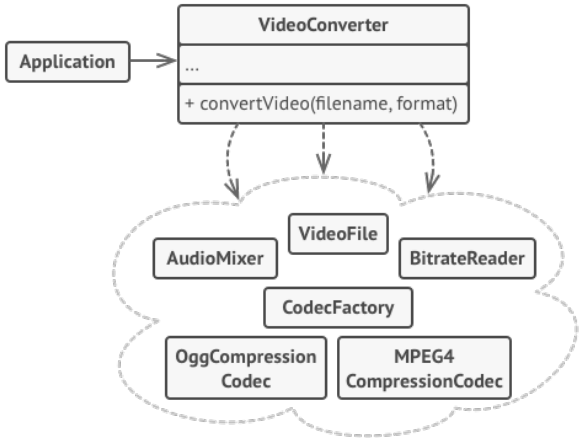
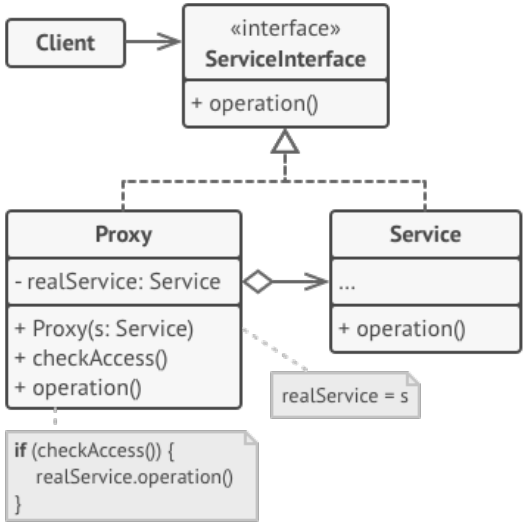
Pattern	Description	Example	Diagram
Observer	An object, called the subject, maintains a list of its dependents, called observers, and notifies them automatically of any state changes When you need an object (Subject) to notify other objects (Observers) but limit dependencies. Subject doesn't know the concrete types of observers	Example: A game wants to broadcast to all systems when the player has died. The gameplay object is the subject, broadcasting information out to several observers (screens, high score, save system)	

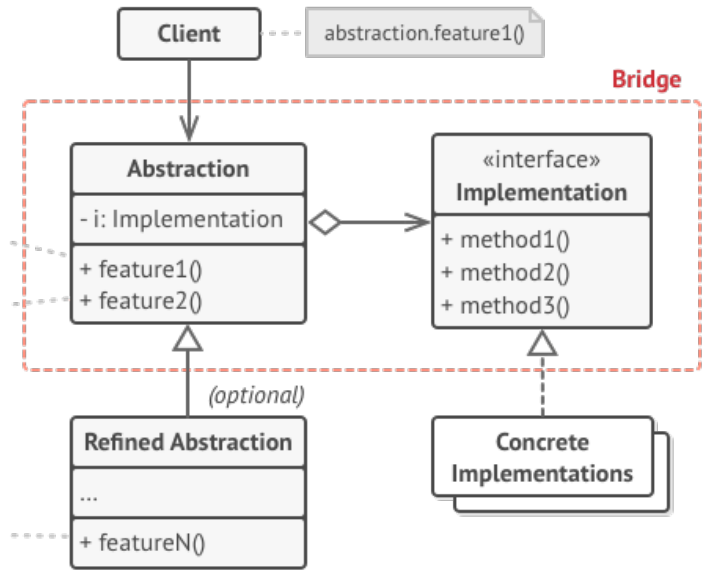
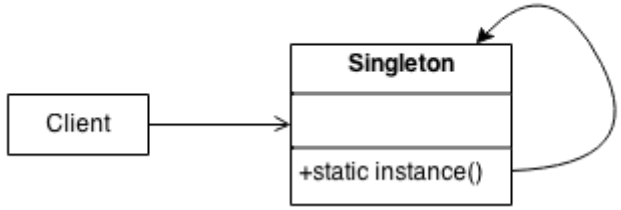
Chain of Responsibility	When a request needs to go through a series of steps. Decouples the sequencing of these steps and makes them re-usable	Example: A document needs to go through several stages of validation. Validate courses correct, fees paid, student information correct.	

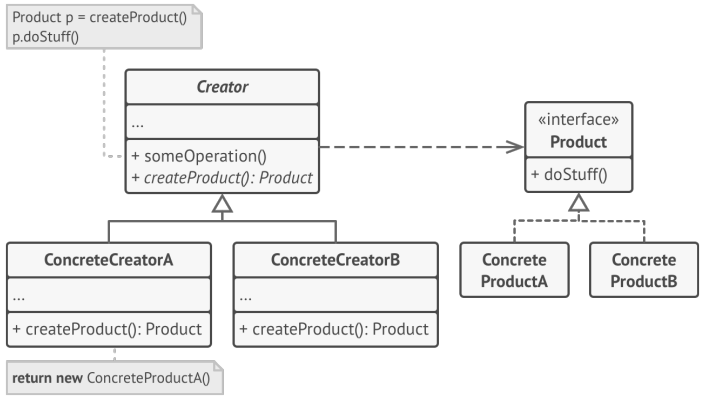
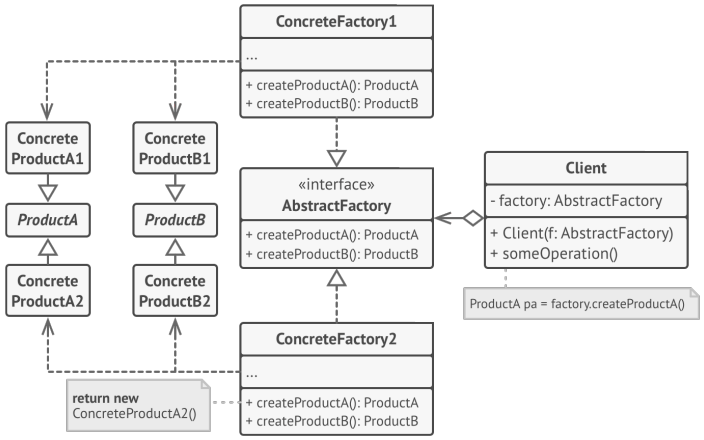
State	When the same object can take on different behaviours depending on its “state”. Having different specialized behaviours within the same class	Example: Game with that shows different screens depending on the game’s states. MainMenu, Gameplay, Gameover	
Strategy	When you need to switch behaviours at runtime Another way of phrasing this is that it let’s you swap algorithms at run-time.	Example: A game character wants to use different weapons. The weapon class contains an attack function. Can swap out the weapon with different objects (axe, sword, hammer) to change the character’s attack behavior	

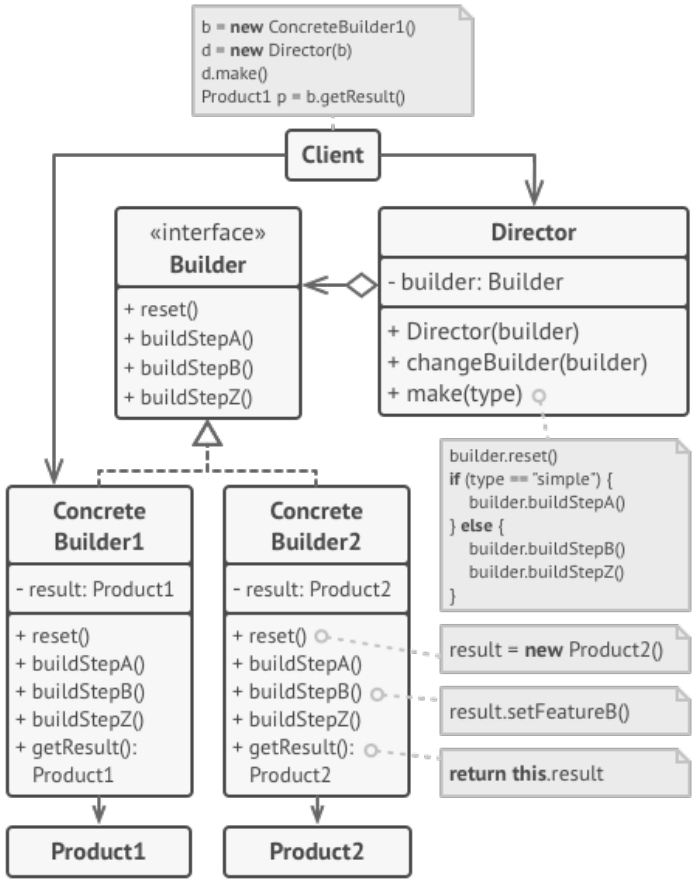
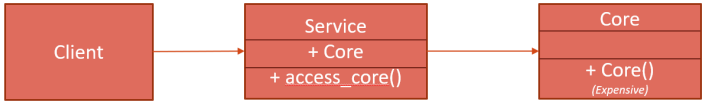
Mediator	mediator pattern defines an object that encapsulates how a set of objects interact When an object needs to send a message, they blindly notify mediator. Mediator object retrieves the notification and knows how where to code to execute next	Example: UI where a bunch of UI components notify where they have been activated. The mediator sends messages from one UI component to the next	

Decorator	Allows behavior to be added to an individual object, dynamically, without affecting the behavior of other objects from the same class When you want to dynamically add behavior to an object without going back to re-write the object.	Example: Donut object where we can add any number and type of toppings to it	

Façade	Encapsulates complex system or API into a simplified interface. Client only interacts with the façade instead of complex systems When you want to hide the complexity of a system from the client, but allow them to use it in a simplified form.	Example: Video converter. You only want to expose the convert video function to the client. But behind the scenes, there are many systems your facade must call to perform the action	
Proxy	A class functioning as an interface to something else. A proxy is a wrapper or agent object that is being called by the client to access the real serving object behind the scenes When you have a real service or object that you wish to hide the client from directly interacting with.	Example: Youtube database is a real service. You don't want all users in the word to access one database. Create a proxy object that intercepts requests and forwards to Youtube database. Repeat requests are cached	

Bridge	The bridge pattern allows the Abstraction and the Implementation to be developed independently and the client code can access only the Abstraction part without being concerned about the Implementation part When you want to break down a large class into smaller components. These components are connected to the main abstraction class via a bridge (instance of the implementation) .	Example: Creating a TV remote classes that controls a Device	
Singleton	Ensure a class has only one instance, and provide a global point of access to it. Use when an application needs one and only one instance of an object that is globally accessible	Example: Save system in a game. The save system can be accessed by any screen, controller, object in the game.	

Factory	When you want to separate creation code from client code. When you want a client or service to only depend on a base class for object creation and not each concrete class.	Example: Forum that needs access to different types of users (Guest, member). Factory can produce both types and the forum uses them without knowing the concrete type	
Abstract Factory	A more evolved form of the Factory Pattern Allows for the creation of a group of related objects . When you want to create different variants of families of objects	Example: Client want to order different themes of furniture. Client passes in the appropriate Factory and get the set of themed furniture.	

Builder	Designed to provide a flexible solution to various object creation problems in object-oriented programming. The intent of the Builder design pattern is to separate the construction of a complex object from its representation The code typically written in the initializer of an object is moved into a separate class	Example: Want to create different types of pizza, but the client doesn't want to manually specify all ingredients. They want to order from a menu. The menu tells the builder to build a very specific type of pizza, and returns that to the client	
Lazy Initialization	**initialization** of an object occurs only when it is actually needed and not before to preserve simplicity of usage and improve performance.	Example: Screen in a program that isn't instantiated until it's actually needed